Experiment 11: CNN for Image Classification

1. Aim

To build and train a Convolutional Neural Network (CNN) using TensorFlow and Keras to classify images into different categories and evaluate its performance using accuracy, loss curves, a confusion matrix, and feature maps.

2. Architecture Diagram



Formulas

3. Main Formulas

A. Convolution operation

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

Where:

- X = Input image.
- K = Filter or kernel.
- Y = Output feature map.

B. ReLU activation

$$f(x) = \max(0, x)$$

It converts negative values into zero and keeps positive values unchanged.

C. Max Pooling

$$P = \max(\text{values in the pooling window})$$

It selects the maximum value from a small region.

D. Softmax function

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

It converts model outputs into class probabilities.

E. Categorical Cross-Entropy Loss

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

It measures the difference between actual class labels and predicted probabilities.

F. Accuracy

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

Code:

```
import time
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# 1. Display runtime information
print("TensorFlow version:", tf.__version__)
print("GPU available:", len(tf.config.list_physical_devices("GPU")) >
0)
```

```
# 2. Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.cifar10.load_data()

# Use a small subset for faster training
x_train = x_train[:2000].astype("float32") / 255.0
y_train = y_train[:2000].flatten()

x_test = x_test[:500].astype("float32") / 255.0
y_test = y_test[:500].flatten()

class_names = [
    "Airplane", "Automobile", "Bird", "Cat", "Deer",
    "Dog", "Frog", "Horse", "Ship", "Truck"
]

print("Training images:", x_train.shape)
print("Testing images:", x_test.shape)

# 3. Build a simple CNN model
model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),

    layers.Conv2D(16, (3, 3), activation="relu",
                  padding="same", name="conv1"),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(32, (3, 3), activation="relu",
                  padding="same", name="conv2"),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(32, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# 4. Train the CNN
start = time.time()
```

```
history = model.fit(
    x_train, y_train,
    epochs=2,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)

print("Training time:", round(time.time() - start, 2), "seconds")

# 5. Evaluate the model
test_loss, test_accuracy = model.evaluate(
    x_test, y_test, verbose=0
)

print("\nTest Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")

# 6. Plot training and validation accuracy/loss
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], marker="o", label="Training")
plt.plot(history.history["val_accuracy"], marker="o",
label="Validation")
plt.title("CNN Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], marker="o", label="Training")
plt.plot(history.history["val_loss"], marker="o", label="Validation")
plt.title("CNN Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

# 7. Predict test images
predictions = model.predict(x_test, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)

# 8. Display confusion matrix
cm = confusion_matrix(y_test, predicted_labels, labels=range(10))
```

```
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)

fig, ax = plt.subplots(figsize=(10, 10))
display.plot(ax=ax, xticks_rotation=45, colorbar=False)
plt.title("CNN Confusion Matrix")
plt.tight_layout()
plt.show()

# 9. Display some misclassified images
wrong_indices = np.where(predicted_labels != y_test)[0]

if len(wrong_indices) > 0:
    count = min(6, len(wrong_indices))
    plt.figure(figsize=(12, 5))

    for i, idx in enumerate(wrong_indices[:count]):
        plt.subplot(2, 3, i + 1)
        plt.imshow(x_test[idx])
        plt.title(
            "True: " + class_names[y_test[idx]] +
            "\nPredicted: " + class_names[predicted_labels[idx]]
        )
        plt.axis("off")

    plt.suptitle("Misclassified Images")
    plt.tight_layout()
    plt.show()
else:
    print("No misclassified images in this test subset.")

# 10. Visualize feature maps from the first convolution layer
feature_model = tf.keras.Model(
    inputs=model.inputs,
    outputs=model.get_layer("conv1").output
)

feature_maps = feature_model.predict(x_test[:1], verbose=0)

plt.figure(figsize=(10, 5))

for i in range(min(8, feature_maps.shape[-1])):
    plt.subplot(2, 4, i + 1)
    plt.imshow(feature_maps[0, :, :, i], cmap="viridis")
    plt.title("Filter " + str(i + 1))
    plt.axis("off")
```

```
plt.suptitle("First Convolution Layer Feature Maps")
plt.tight_layout()
plt.show()

print("\nExperiment 11 completed successfully!")
```

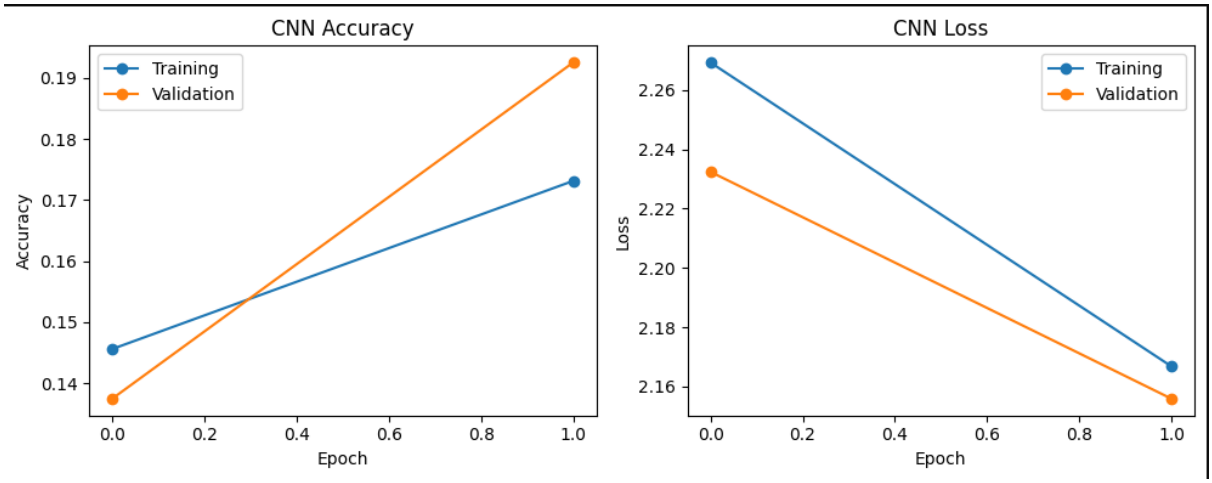
```
output:
Training images: (2000, 32, 32, 3)
Testing images: (500, 32, 32, 3)
```

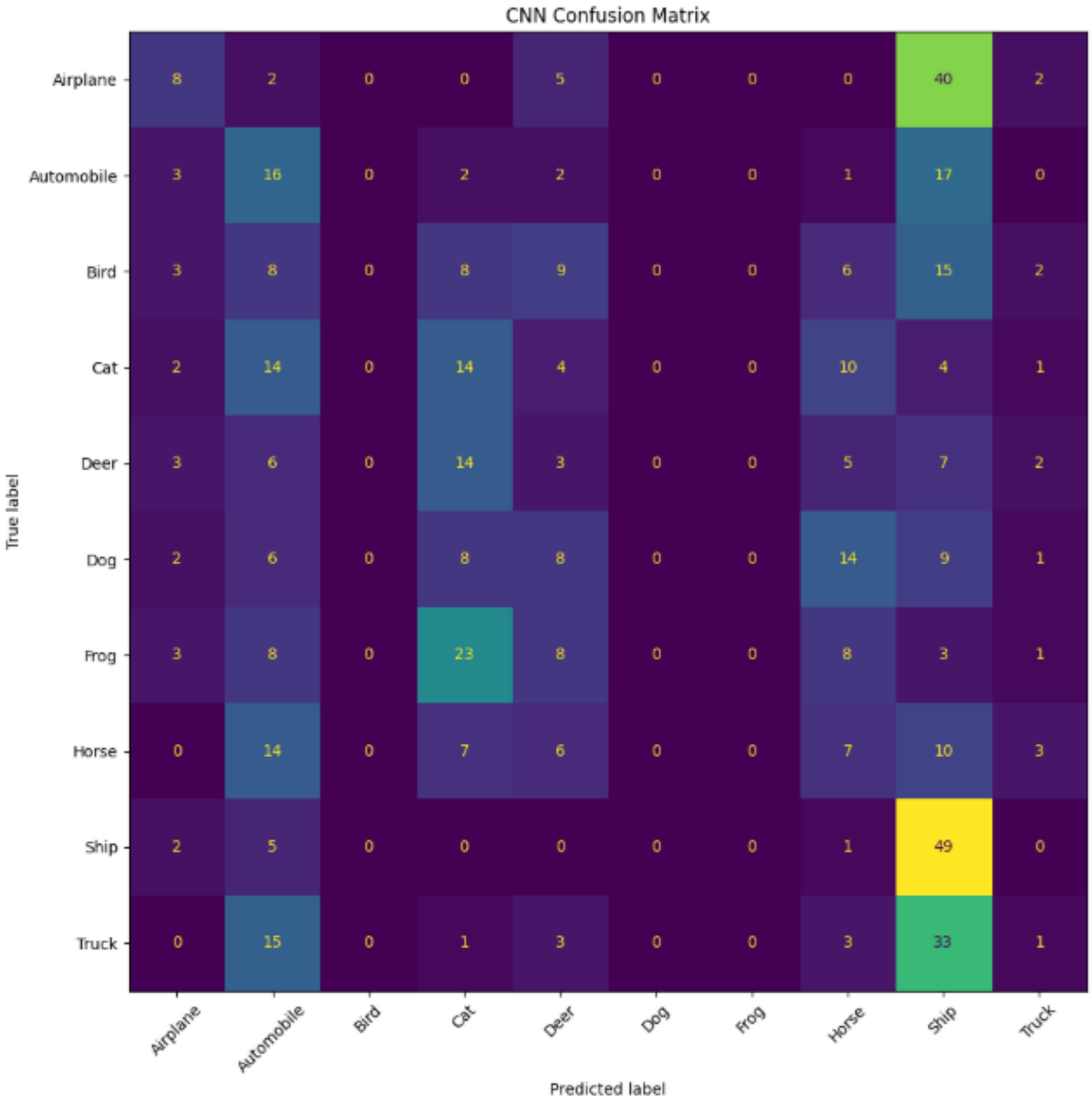
Model: "sequential"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 32, 32, 16)	448
max_pooling2d (MaxPooling2D)	(None, 16, 16, 16)	0
conv2 (Conv2D)	(None, 16, 16, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 32)	65,568
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 10)	330

Total params: 70,986 (277.29 KB)
Trainable params: 70,986 (277.29 KB)
Non-trainable params: 0 (0.00 B)

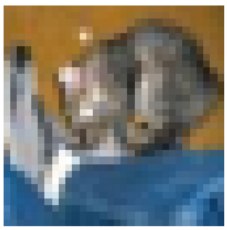
```
Test Loss: 2.1308
Test Accuracy: 19.6 %
```





Misclassified Images

True: Cat
Predicted: Automobile



True: Airplane
Predicted: Ship



True: Frog
Predicted: Horse



True: Frog
Predicted: Cat

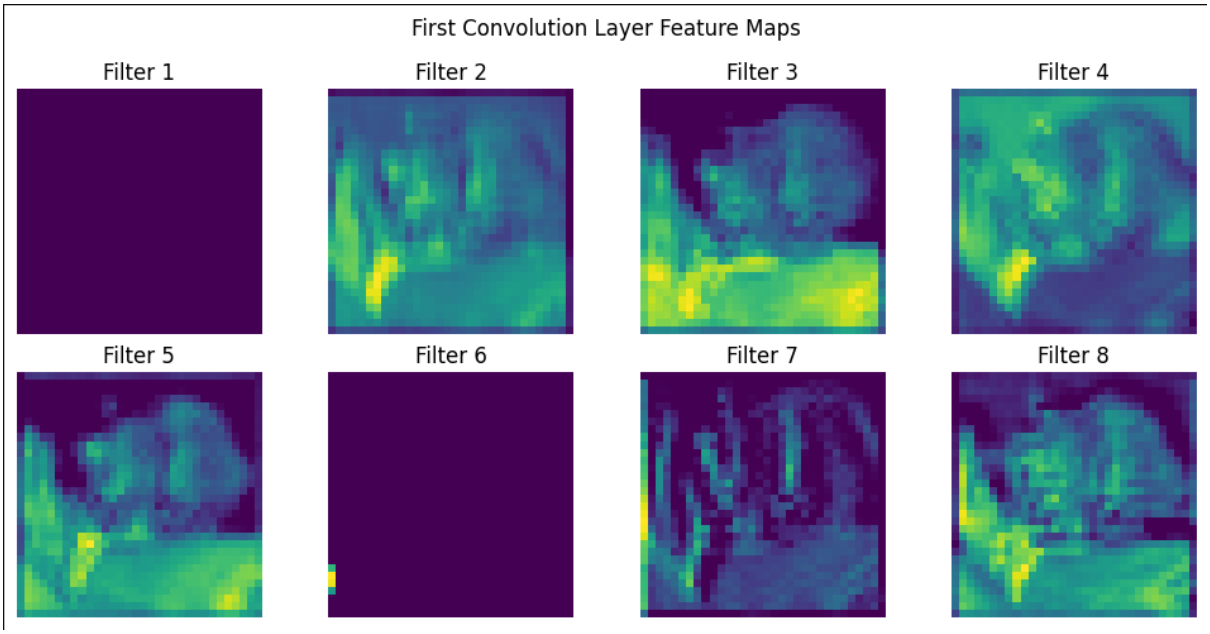


True: Automobile
Predicted: Horse



True: Frog
Predicted: Deer



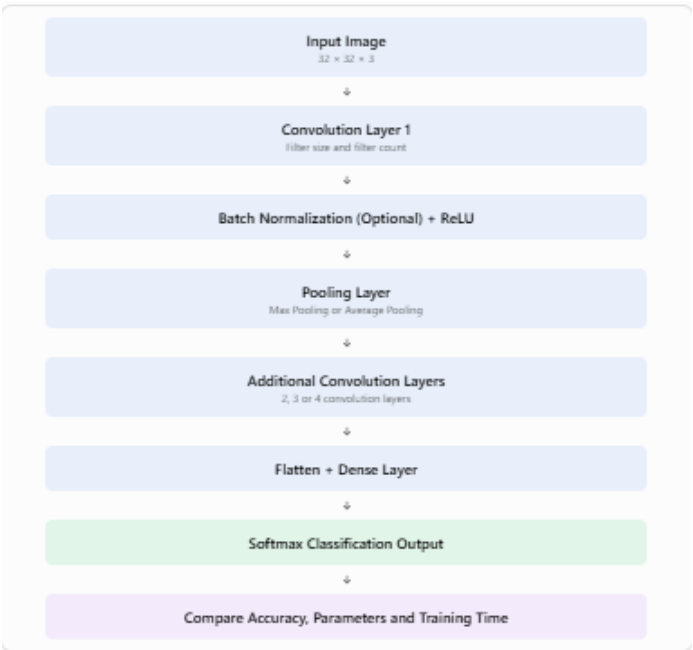


Experiment 12: CNN Architecture and Hyperparameter Comparison

1. Aim

To compare different CNN architectures and hyperparameters by varying convolution layers, filter sizes, filter counts, pooling methods, batch normalization, learning rates, and batch sizes, and evaluate their accuracy, parameter count, and training time.

2. Architecture Diagram



Formula

A. Convolution output size

$$O = \frac{N - F + 2P}{S} + 1$$

Where:

- N = Input size.
- F = Filter size.
- P = Padding.
- S = Stride.

B. Number of convolution parameters

$$Parameters = (F_h \times F_w \times C_{in} + 1) \times C_{out}$$

Where:

- F_h, F_w = Filter height and width.
- C_{in} = Input channels.
- C_{out} = Number of filters.

C. Batch Normalization

$$BN(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

It normalizes layer outputs to help stabilize training.

D. Learning rate update

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

Where η is the learning rate.

E. Accuracy

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

F. Training time

$$Training\ Time = End\ Time - Start\ Time$$

Code:

```
import time
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
print("Loading CIFAR-10 dataset...")

(x_train, y_train), (x_test, y_test) = (
    tf.keras.datasets.cifar10.load_data()
)
x_train = x_train[:1200].astype("float32") / 255.0
y_train = y_train[:1200].flatten()

x_test = x_test[:300].astype("float32") / 255.0
```

```
y_test = y_test[:300].flatten()

print("Training images:", len(x_train))
print("Testing images:", len(x_test))
def build_cnn(num_layers=2, filter_size=3, filters=16,
              pooling="max", batch_norm=False):

    model = models.Sequential()
    model.add(layers.Input(shape=(32, 32, 3)))

    for i in range(num_layers):
        number_of_filters = filters * (2 ** i)

        model.add(layers.Conv2D(
            number_of_filters,
            (filter_size, filter_size),
            padding="same",
            use_bias=not batch_norm
        ))

        if batch_norm:
            model.add(layers.BatchNormalization())

        model.add(layers.Activation("relu"))

        if pooling == "max":
            model.add(layers.MaxPooling2D((2, 2)))
        else:
            model.add(layers.AveragePooling2D((2, 2)))

    model.add(layers.Flatten())
    model.add(layers.Dense(32, activation="relu"))
    model.add(layers.Dense(10, activation="softmax"))

    return model
configs = [
    {"name": "Baseline", "layers": 2, "size": 3,
     "filters": 16, "pool": "max", "bn": False,
     "lr": 0.001, "batch": 64},

    {"name": "3 Conv layers", "layers": 3, "size": 3,
     "filters": 16, "pool": "max", "bn": False,
     "lr": 0.001, "batch": 64},

    {"name": "4 Conv layers", "layers": 4, "size": 3,
     "filters": 16, "pool": "max", "bn": False,
     "lr": 0.001, "batch": 64},
```

```
    {"name": "5x5 filters", "layers": 2, "size": 5,
     "filters": 16, "pool": "max", "bn": False,
     "lr": 0.001, "batch": 64},

    {"name": "Fewer filters", "layers": 2, "size": 3,
     "filters": 8, "pool": "max", "bn": False,
     "lr": 0.001, "batch": 64},

    {"name": "Average pooling", "layers": 2, "size": 3,
     "filters": 16, "pool": "avg", "bn": False,
     "lr": 0.001, "batch": 64},

    {"name": "Batch normalisation", "layers": 2, "size": 3,
     "filters": 16, "pool": "max", "bn": True,
     "lr": 0.001, "batch": 64},

    {"name": "LR and batch changed", "layers": 2, "size": 3,
     "filters": 16, "pool": "max", "bn": False,
     "lr": 0.0005, "batch": 32}
]

# 4. TRAIN AND EVALUATE EACH CONFIGURATION
results = []

for config in configs:
    print("\nRunning:", config["name"])

    tf.keras.backend.clear_session()

    model = build_cnn(
        num_layers=config["layers"],
        filter_size=config["size"],
        filters=config["filters"],
        pooling=config["pool"],
        batch_norm=config["bn"]
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=config["lr"]
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    parameter_count = model.count_params()

    start_time = time.time()
```

```
    model.fit(
        x_train, y_train,
        epochs=2,
        batch_size=config["batch"],
        verbose=0
    )

    training_time = time.time() - start_time

    test_loss, test_accuracy = model.evaluate(
        x_test, y_test, verbose=0
    )

    results.append({
        "Configuration": config["name"],
        "Conv Layers": config["layers"],
        "Filter Size": str(config["size"]) + "x" + str(config["size"]),
        "Initial Filters": config["filters"],
        "Pooling": config["pool"],
        "Batch Norm": config["bn"],
        "Learning Rate": config["lr"],
        "Batch Size": config["batch"],
        "Accuracy (%)": round(test_accuracy * 100, 2),
        "Parameters": parameter_count,
        "Training Time (s)": round(training_time, 2)
    })

    print("Accuracy:", round(test_accuracy * 100, 2), "%")
    print("Parameters:", parameter_count)
    print("Training time:", round(training_time, 2), "seconds")
results_df = pd.DataFrame(results)

print("\nCNN CONFIGURATION COMPARISON")
print(results_df.to_string(index=False))
plt.figure(figsize=(12, 5))
plt.bar(results_df["Configuration"], results_df["Accuracy (%)"])
plt.title("CNN Test Accuracy Comparison")
plt.xlabel("Configuration")
plt.ylabel("Test Accuracy (%)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 5))
plt.bar(results_df["Configuration"], results_df["Parameters"])
plt.title("CNN Model Size Comparison")
plt.xlabel("Configuration")
plt.ylabel("Trainable Parameters")
```

```
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 5))
plt.bar(results_df["Configuration"],
        results_df["Training Time (s)"])
plt.title("CNN Training Time Comparison")
plt.xlabel("Configuration")
plt.ylabel("Training Time (seconds)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
best_row = results_df.loc[
    results_df["Accuracy (%)"].idxmax()
]

print("\nHighest test accuracy configuration:")
print(best_row.to_string())

# Save results for the practical record
results_df.to_csv("experiment12_results.csv", index=False)
print("\nResults saved to experiment12_results.csv")
print("Experiment 12 completed successfully!")

Training images: 1200
Testing images: 300

Running: Baseline
Accuracy: 18.0 %
Parameters: 70986
Training time: 8.88 seconds

Running: 3 Conv layers
Accuracy: 22.33 %
Parameters: 56714
Training time: 6.19 seconds

Running: 4 Conv layers
Accuracy: 19.67 %
Parameters: 114186
Training time: 6.77 seconds

Running: 5x5 filters
Accuracy: 24.33 %
Parameters: 79946
Training time: 5.84 seconds

Running: Fewer filters
Accuracy: 20.67 %
Parameters: 34522
Training time: 5.83 seconds

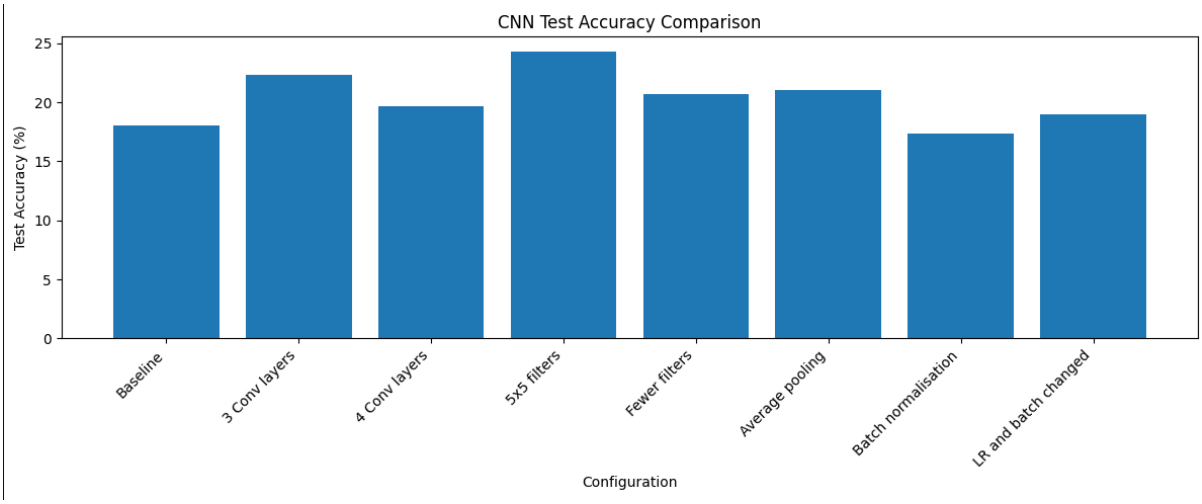
Running: Average pooling
```

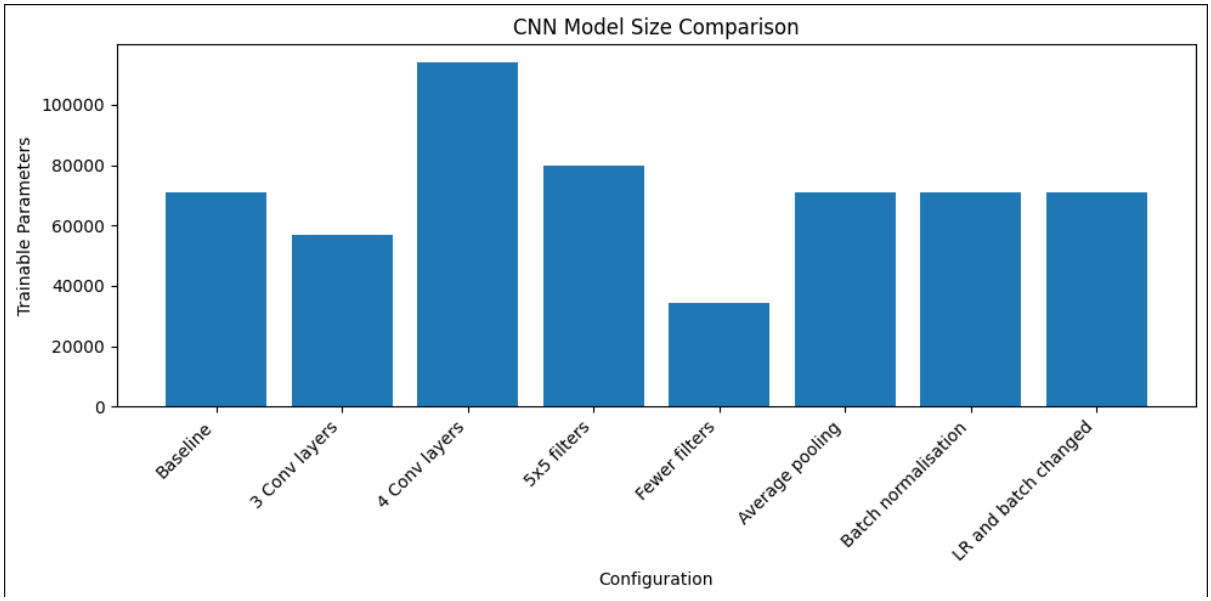
```
Accuracy: 21.0 %
Parameters: 70986
Training time: 3.92 seconds

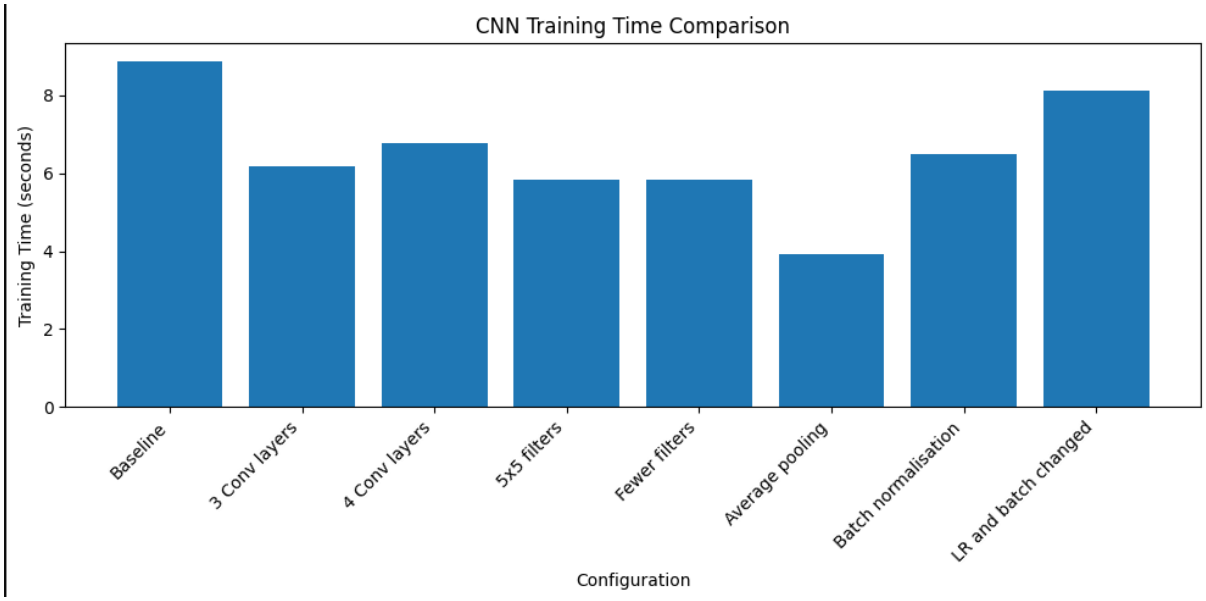
Running: Batch normalisation
Accuracy: 17.33 %
Parameters: 71130
Training time: 6.5 seconds

Running: LR and batch changed
Accuracy: 19.0 %
Parameters: 70986
Training time: 8.13 seconds

CNN CONFIGURATION COMPARISON
       Configuration  Conv Layers Filter Size  Initial Filters Pooling  Batch Norm
Learning Rate  Batch Size  Accuracy (%)  Parameters  Training Time (s)
            Baseline            2         3x3               16     max       False
0.0010          64         18.00       70986               8.88
       3 Conv layers            3         3x3               16     max       False
0.0010          64         22.33       56714               6.19
       4 Conv layers            4         3x3               16     max       False
0.0010          64         19.67      114186               6.77
         5x5 filters            2         5x5               16     max       False
0.0010          64         24.33       79946               5.84
       Fewer filters            2         3x3                8     max       False
0.0010          64         20.67       34522               5.83
     Average pooling            2         3x3               16     avg       False
0.0010          64         21.00       70986               3.92
 Batch normalisation            2         3x3               16     max        True
0.0010          64         17.33       71130               6.50
LR and batch changed            2         3x3               16     max       False
0.0005          32         19.00       70986               8.13
```





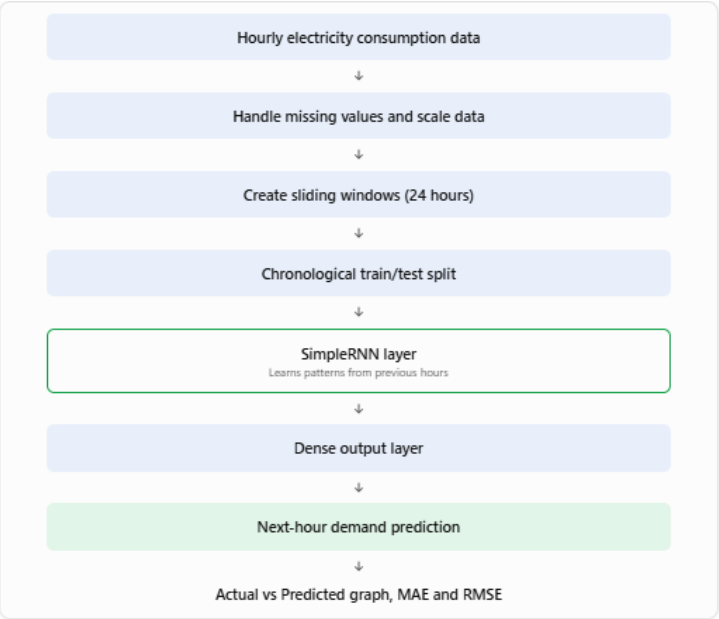


Experiment 13: Sequential Dataset Preparation and RNN Prediction

1. Aim

To prepare time-series data using sliding windows, build and train a SimpleRNN model, predict next-hour electricity demand, and evaluate its performance using MAE and RMSE.

2. Architecture diagram



3. Main formulas

A. Sliding window

$$X_t = [x_{t-24}, ..., x_{t-1}]$$

$$Y_t = x_t$$

This means the previous 24 hours are used to predict the next hour.

B. RNN hidden state

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$$

C. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i-1}^{n}|y_i - \hat{y}_i|$$

D. Root Mean Square Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i-1}^{n}(y_i - \hat{y}_i)^2}$$

Where y_i is actual demand and $\hat{y}_i$ is predicted demand.

code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
np.random.seed(42)
hours = 24 * 365
time = np.arange(hours)
demand = (
    100
    + 15 * np.sin(2 * np.pi * time / 24)
    + 10 * np.sin(2 * np.pi * time / (24 * 365))
    + np.random.normal(0, 3, hours)
)
df = pd.DataFrame({"Demand": demand})
df["Demand"] = df["Demand"].interpolate()
split_point = int(len(df) * 0.8)
scaler = MinMaxScaler()
scaler.fit(df[["Demand"]].iloc[:split_point])
scaled_data = scaler.transform(df[["Demand"]])
window_size = 24
X, y, target_indices = [], [], []
for i in range(window_size, len(scaled_data)):
    X.append(scaled_data[i-window_size:i])
    y.append(scaled_data[i])
    target_indices.append(i)

X = np.array(X)
y = np.array(y)
target_indices = np.array(target_indices)
train_mask = target_indices < split_point
```

```
test_mask = target_indices >= split_point
X_train, y_train = X[train_mask], y[train_mask]
X_test, y_test = X[test_mask], y[test_mask]
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
model = Sequential([
    tf.keras.Input(shape=(24, 1)),
    SimpleRNN(32, activation="tanh"),
    Dense(1)
])
model.compile(optimizer="adam", loss="mse")
history = model.fit(
    X_train, y_train,
    epochs=8,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
predicted = model.predict(X_test, verbose=0)
actual_values = scaler.inverse_transform(y_test)
predicted_values = scaler.inverse_transform(predicted)
mae = mean_absolute_error(actual_values, predicted_values)
rmse = np.sqrt(mean_squared_error(actual_values, predicted_values))
print("\nMAE:", round(mae, 4))
print("RMSE:", round(rmse, 4))
plt.figure(figsize=(12, 5))
plt.plot(actual_values[:200], label="Actual Demand")
plt.plot(predicted_values[:200], label="Predicted Demand")
plt.title("Actual vs Predicted Electricity Demand")
plt.xlabel("Test Hours")
plt.ylabel("Electricity Demand")
```

plt.legend()

plt.grid()

plt.show()

plt.figure(figsize=(8, 4))

plt.plot(history.history["loss"], label="Training Loss")

plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("RNN Training Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

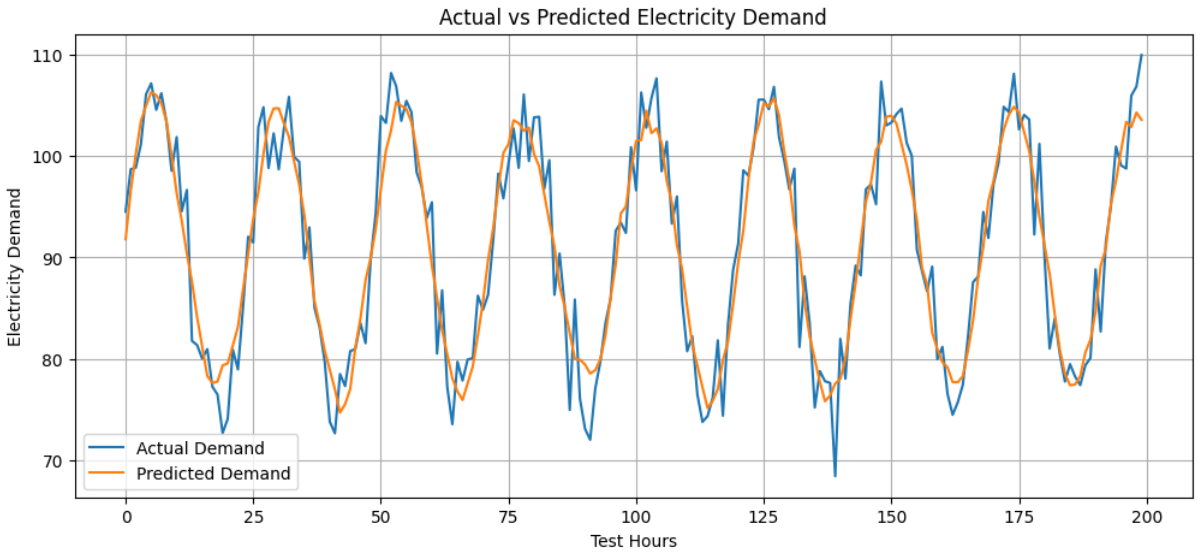
plt.grid()

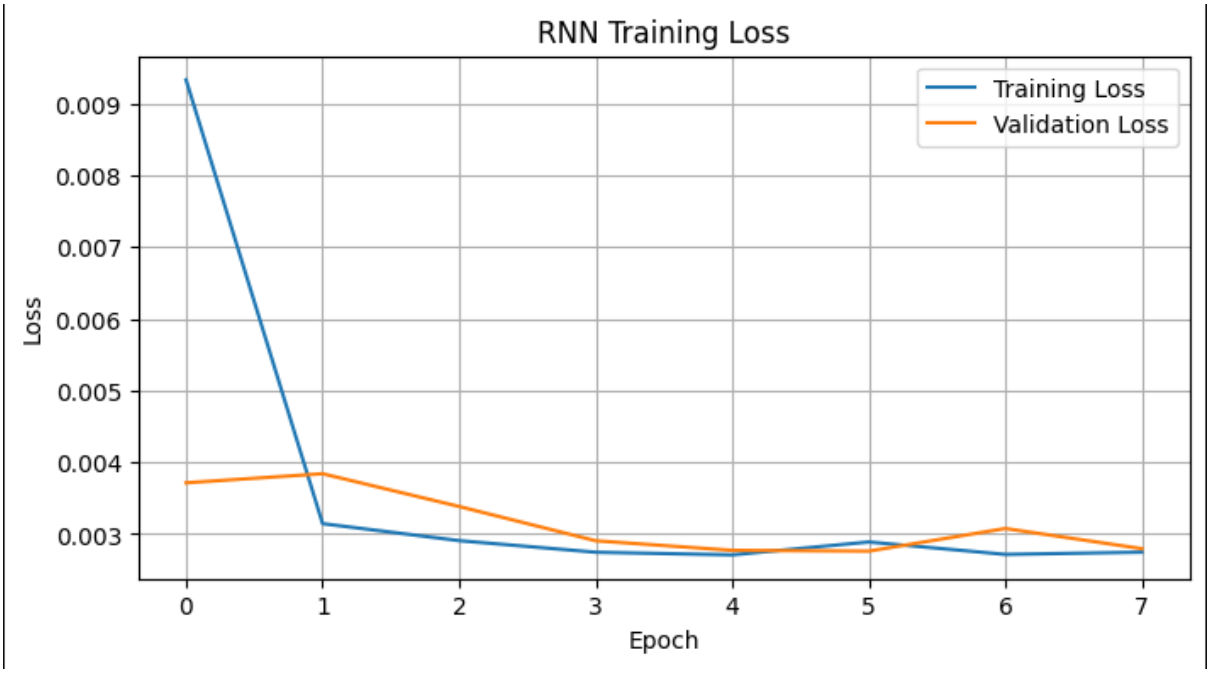
plt.show()

code:

```
Training data: (6984, 24, 1)
Testing data: (1752, 24, 1)
```

```
MAE: 2.7027
RMSE: 3.4017
```



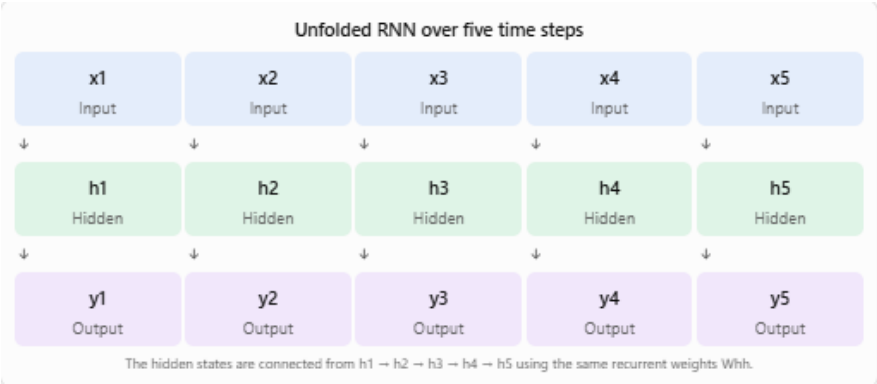


Experiment 14: Unfolded RNN Architecture Visualization

1. Aim

To visualize a SimpleRNN unfolded across five time steps, manually calculate hidden states, compare them with Keras outputs, and display the network using NetworkX.

2. Architecture diagram



3.formula

A. Hidden state calculation

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b)$$

B. Output calculation

$$y_t = h_t W_{hy} + b_y$$

C. Initial hidden state

$$h_0 = [0, 0, 0]$$

code:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import networkx as nx

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, SimpleRNN, Dense

np.random.seed(42)
tf.random.set_seed(42)

words = ["I", "love", "machine", "learning", "today"]

X = np.array([[
    [0.2, 0.5],
    [0.4, 0.1],
    [0.7, 0.3],
    [0.6, 0.8],
    [0.3, 0.9]
]], dtype=np.float32)

inputs = Input(shape=(5, 2))

rnn_layer = SimpleRNN(
    3,
    activation="tanh",
    return_sequences=True,
    name="simple_rnn"
)

hidden_output = rnn_layer(inputs)
output_layer = Dense(1, name="output_layer")
outputs = output_layer(hidden_output)

model = Model(inputs, outputs)

keras_hidden = rnn_layer(X)

print("Keras Hidden States:")
print(keras_hidden.numpy()[0])
Wxh, Whh, bias = rnn_layer.get_weights()

Why, by = output_layer.get_weights()

print("\nInput Weight Shape:", Wxh.shape)
```

```
print("Recurrent Weight Shape:", Whh.shape)
print("Output Weight Shape:", Why.shape)

h_previous = np.zeros((1, 3))
manual_hidden = []

for t in range(5):
    x_t = X[:, t, :]

    h_current = np.tanh(
        np.dot(x_t, Wxh)
        + np.dot(h_previous, Whh)
        + bias
    )

    manual_hidden.append(h_current[0])
    h_previous = h_current

manual_hidden = np.array(manual_hidden)

print("\nManually Calculated Hidden States:")
print(manual_hidden)
difference = np.max(
    np.abs(manual_hidden - keras_hidden.numpy()[0])
)

print("\nMaximum Difference:", difference)
manual_output = np.dot(manual_hidden, Why) + by

print("\nOutput Values:")
print(manual_output.flatten())
G = nx.DiGraph()

for t in range(1, 6):
    G.add_node(f"x{t}")
    G.add_node(f"h{t}")
    G.add_node(f"y{t}")

    G.add_edge(f"x{t}", f"h{t}", label="Wxh")
    G.add_edge(f"h{t}", f"y{t}", label="Why")

    if t > 1:
        G.add_edge(f"h{t-1}", f"h{t}", label="Whh")
pos = {}

for t in range(1, 6):
    pos[f"x{t}"] = (t, 2)
    pos[f"h{t}"] = (t, 1)
```

```
    pos[f"y{t}"] = (t, 0)

plt.figure(figsize=(13, 5))

nx.draw(
    G, pos,
    with_labels=True,
    node_size=1800,
    node_color="lightblue",
    font_size=11,
    arrows=True,
    arrowsize=18
)

edge_labels = nx.get_edge_attributes(G, "label")
nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels=edge_labels,
    font_size=9
)

plt.title("Unfolded SimpleRNN Architecture")
plt.axis("off")
plt.show()
plt.figure(figsize=(9, 5))

for i in range(3):
    plt.plot(
        range(1, 6),
        manual_hidden[:, i],
        marker="o",
        label=f"Hidden Unit {i+1}"
    )

plt.title("Hidden State Values Across Time Steps")
plt.xlabel("Time Step")
plt.ylabel("Hidden State Value")
plt.xticks(range(1, 6))
plt.legend()
plt.grid()
plt.show()

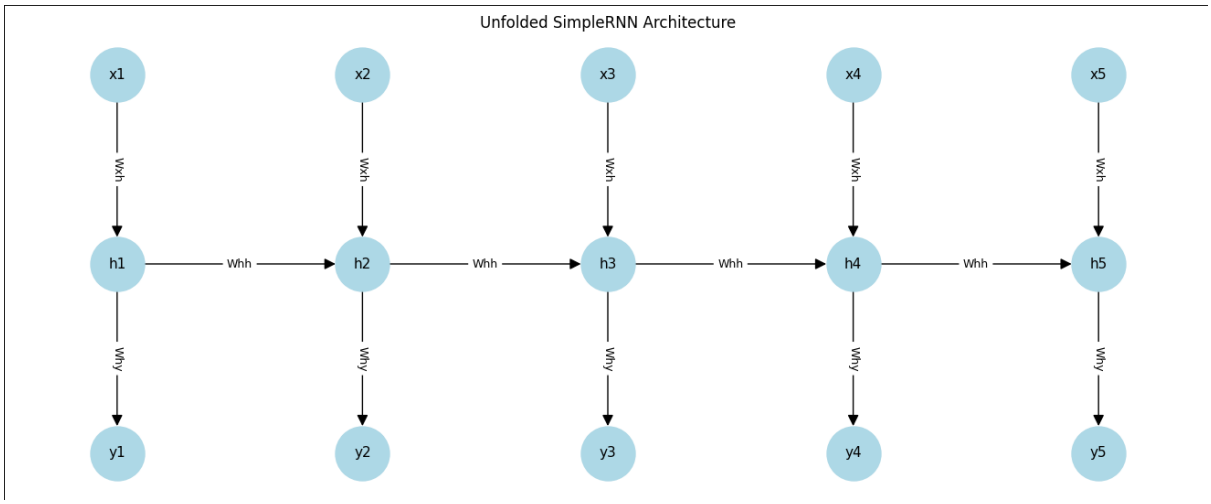
output:
Keras Hidden States:
[[ 0.38540542  0.4282115  -0.07485244]
 [-0.2771418   0.532447    0.576793  ]
 [ 0.63759524  0.66463274  0.8359957 ]
 [ 0.31771353  0.47367015  0.8370034 ]
 [ 0.58045006  0.279121    0.6055201 ]]
```

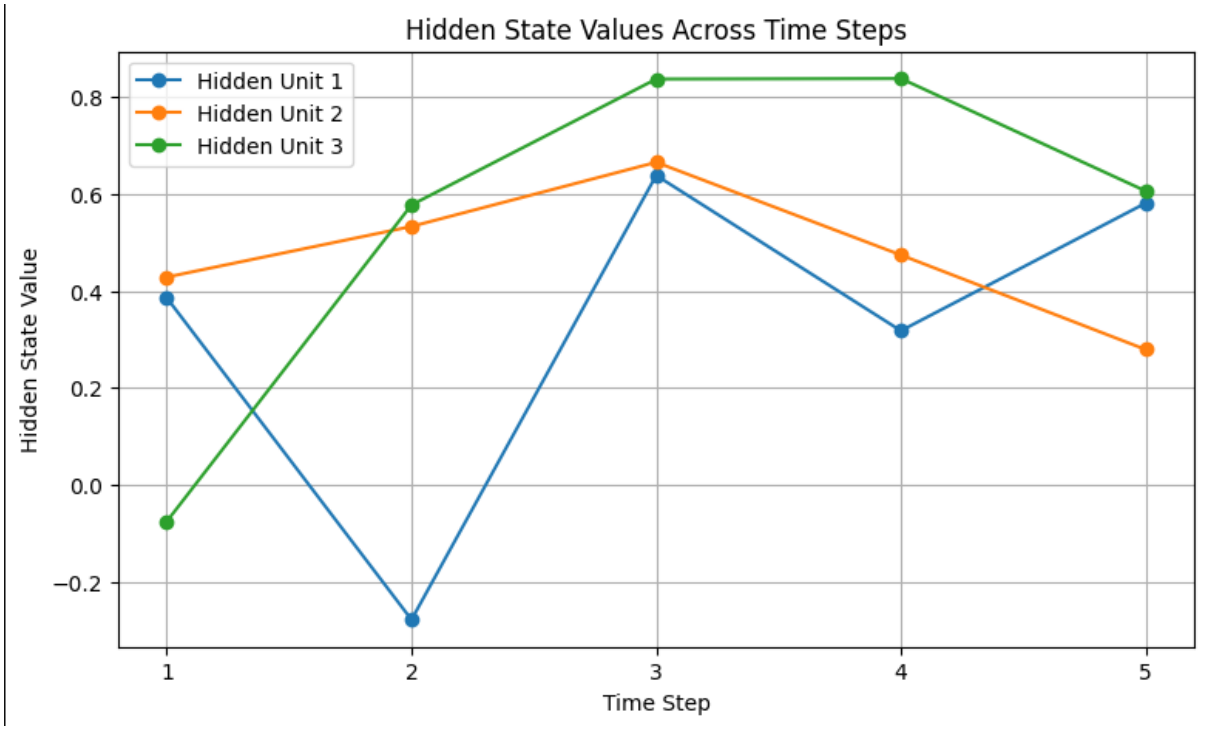
```
Input Weight Shape: (2, 3)
Recurrent Weight Shape: (3, 3)
Output Weight Shape: (3, 1)

Manually Calculated Hidden States:
[[ 0.38540542  0.42821155 -0.07485245]
 [-0.27714185  0.53244701  0.576793  ]
 [ 0.63759522  0.66463275  0.83599565]
 [ 0.3177136   0.47367025  0.83700333]
 [ 0.58044998  0.2791211   0.60552008]]

Maximum Difference: 9.303652209746716e-08

Output Values:
[-0.15555835  0.07180025 -0.76471305 -0.55907732 -0.70504342]
```



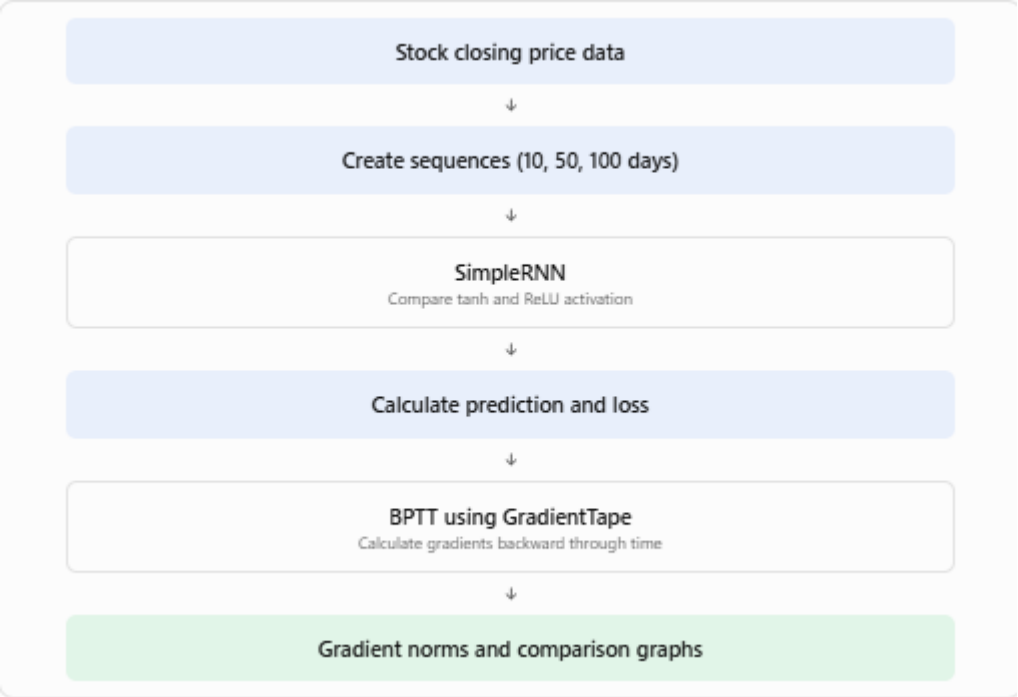


Experiment 15: BPTT Flow and Gradient Analysis

1. Aim

To analyze gradient flow in a SimpleRNN using Backpropagation Through Time (BPTT), compare tanh and ReLU activation functions, and identify vanishing and exploding gradients for different sequence lengths.

2. Architecture diagram



3. Main formulas

A. RNN hidden state

$$h_t - f(W_{xh}x_t + W_{hh}h_{t-1} + b)$$

B. Loss function (Mean Squared Error)

$$L - \frac{1}{n}\sum_{i-1}^{n}(y_i - \hat{y}_i)^2$$

C. Backpropagation Through Time

$$\frac{\partial L}{\partial h_t} - \frac{\partial L}{\partial h_T}\prod_{k-t+1}^{T}\frac{\partial h_k}{\partial h_{k-1}}$$

This formula shows how gradients travel backward through previous time steps.

D. Gradient norm

$$||G||_2 - \sqrt{\sum_{i-1}^{n} g_i^2}$$

Code:

```
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.layers import SimpleRNN, Dense

np.random.seed(42)
tf.random.set_seed(42)
days = 3 * 252
time = np.arange(days)

prices = (
    100
    + 0.03 * time
    + 5 * np.sin(2 * np.pi * time / 30)
    + np.random.normal(0, 1, days)
)
prices = (prices - prices.min()) / (prices.max() - prices.min())
sequence_lengths = [10, 50, 100]
activations = ["tanh", "relu"]

units = 8
gradient_results = {}
for activation_name in activations:

    for seq_len in sequence_lengths:
        X = tf.constant(
            prices[:seq_len].reshape(1, seq_len, 1),
            dtype=tf.float32
        )

        target = tf.constant(
            [[prices[seq_len]]],
            dtype=tf.float32
        )
        rnn = SimpleRNN(
            units,
            activation=activation_name,
            return_sequences=True,
            kernel_initializer="glorot_uniform",
            recurrent_initializer="glorot_uniform"
        )

        rnn(tf.zeros((1, seq_len, 1)))

        Wxh, Whh, bias = rnn.get_weights()
```

```
Wxh = tf.Variable(Wxh)
Whh = tf.Variable(Whh)
bias = tf.Variable(bias)

dense = Dense(1)
dense(tf.zeros((1, units)))

Why, by = dense.get_weights()

Why = tf.Variable(Why)
by = tf.Variable(by)

with tf.GradientTape() as tape:

    h_previous = tf.zeros((1, units))
    hidden_states = []

    for t in range(seq_len):

        x_t = X[:, t, :]

        z = (
            tf.matmul(x_t, Wxh)
            + tf.matmul(h_previous, Whh)
            + bias
        )

        if activation_name == "tanh":
            h_current = tf.math.tanh(z)
        else:
            h_current = tf.nn.relu(z)

        hidden_states.append(h_current)
        h_previous = h_current
    prediction = tf.matmul(h_previous, Why) + by
    loss = tf.reduce_mean(tf.square(target - prediction))
gradients = tape.gradient(loss, hidden_states)

norms = []

for grad in gradients:
    norm = tf.norm(grad).numpy()
    norms.append(norm)

gradient_results[(activation_name, seq_len)] = norms

print(
```

```
            activation_name,
            "Sequence Length:", seq_len,
            "Loss:", round(loss.numpy(), 6),
            "First Gradient:", round(norms[0], 8),
            "Last Gradient:", round(norms[-1], 8)
        )
plt.figure(figsize=(12, 6))

for activation_name in activations:
    for seq_len in sequence_lengths:

        norms = gradient_results[(activation_name, seq_len)]

        plt.plot(
            range(1, seq_len + 1),
            np.maximum(norms, 1e-12),
            label=f"{activation_name}, Length {seq_len}"
        )

plt.yscale("log")
plt.title("Gradient Flow in RNN using BPTT")
plt.xlabel("Time Step")
plt.ylabel("Gradient Norm (Log Scale)")
plt.legend()
plt.grid()
plt.show()
for activation_name in activations:
    for seq_len in sequence_lengths:

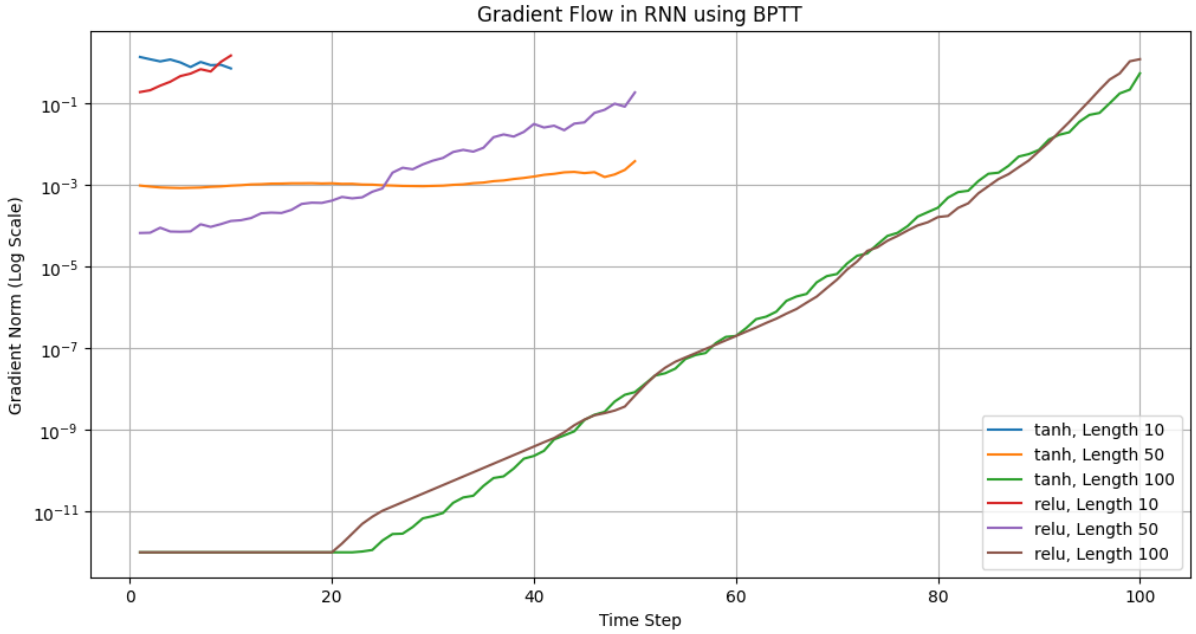
        norms = gradient_results[(activation_name, seq_len)]

        print(
            f"{activation_name}, {seq_len} days: "
            f"Minimum = {min(norms):.8f}, "
            f"Maximum = {max(norms):.8f}"
        )

print("\nGradient analysis completed successfully!")

output:
tanh Sequence Length: 10 Loss: 0.086889 First Gradient: 1.3731108 Last
Gradient: 0.7195583
tanh Sequence Length: 50 Loss: 2e-06 First Gradient: 0.00097029 Last
Gradient: 0.0038425
tanh Sequence Length: 100 Loss: 0.054509 First Gradient: 0.0 Last
Gradient: 0.54563165
relu Sequence Length: 10 Loss: 0.226471 First Gradient: 0.18987405 Last
Gradient: 1.4896886
relu Sequence Length: 50 Loss: 0.007628 First Gradient: 6.687e-05 Last
Gradient: 0.18572775
```

```
relu Sequence Length: 100 Loss: 0.149826 First Gradient: 0.0 Last
Gradient: 1.2122349
```




```
tanh, 10 days: Minimum = 0.71955830, Maximum = 1.37311077
tanh, 50 days: Minimum = 0.00084483, Maximum = 0.00384250
tanh, 100 days: Minimum = 0.00000000, Maximum = 0.54563165
relu, 10 days: Minimum = 0.18987405, Maximum = 1.48968863
relu, 50 days: Minimum = 0.00006687, Maximum = 0.18572773
relu, 100 days: Minimum = 0.00000000, Maximum = 1.21223485

Gradient analysis completed successfully!
```